Working in the Vivado GUI: synthesis, implementation and every kind of simulation

This page explains the error

```
[Synth 8-439] module 'xbar' not found [ssc2_core.v:467]
[Synth 8-6156] failed synthesizing module 'ssc2_core' [ssc2_core.v:40]
[Common 17-69] Command failed: Vivado Synthesis failed
```

why it happens in a project made by hand, and how to get a project in which Run Synthesis, Run Implementation, Generate Bitstream, behavioural, post-synthesis and post-implementation simulation all work.

1. The quick fix

Do not build the project by hand. Double-click

```
windows\0_create_project_gui.bat
```

or, in the Vivado Tcl Console:

```
cd D:/smart_serial_controller_v2
source vivado/create_project.tcl
```

It creates `build\gui\ssc2_gui.xpr` and opens it. Everything in sections 3-5 below is already set up in that project.

2. What went wrong, and why

A Vivado project has separate **file sets**, and each tool only looks at its own set:

file set (Sources window)	used by
Design Sources (`sources_1`)	synthesis, implementation, and every simulation
Constraints (`constrs_1`)	synthesis and implementation (pins, clock)
Simulation Sources (`sim_1`, ...)	simulation only

Problem 1: a design file was added as a simulation source. `xbar.v` was added together with `tb_xbar.v` through *Add Sources -> Add or create simulation sources*. It then lives only in `sim_1`. The behavioural simulation of `tb_xbar` works (it sees `sim_1`), which is why the waveform showed `checks = 1217806, errors = 0`. Synthesis only reads Design Sources, and `ssc2_core.v` line 467 instantiates `xbar`, so synthesis stops with *module 'xbar' not found*. Vivado stops at the first missing module, so other files that are missing too (`rtl/noc`, `rtl/hub` ...) only show up after this one is fixed. The design needs **all 27 files** of `rtl/v1`, `rtl/noc`, `rtl/se`, `rtl/xbar`, `rtl/hub` and `rtl/top` as Design Sources.

Problem 2: the wrong top module. The project's top was `ssc2_axi_top`. That module is the controller as one block *inside the Zynq block design* (Way 2): its ports are an AXI4-Lite slave, an AXI4 master, the interrupt and the pad buses, **410 signals** in total. On its own it is not a board

design: the Zynq-7020 package has only 200 user I/O pins, none of these signals has a pin constraint, and the AXI ports must go to the ARM, not to pins. Even after synthesis worked, implementation would fail at I/O placement (*IO placement is infeasible*) or at bitstream (unconstrained I/O). The two real board tops are:

top	file	what	constraints
`zed2_top_standalone`	`boards/zedboard/zed2_top_standalone.v` (+ `zed2_pads.v`)	Way 1, FPGA only	`zed2_pins.xdc`, `zed2_standalone.xdc`
`zed2_top_ps`	`boards/zedboard/zed2_top_ps.v` (+ `zed2_pads.v`)	Way 2, needs the block design with the Zynq PS	`zed2_pins.xdc`, `zed2_buttons_ps.xdc`; build with `vivado/build_ps_system.tcl`

Problem 3: post-synthesis / post-implementation simulation. These simulations do not simulate the Verilog. They simulate the **netlist** that synthesis or implementation produced (LUTs and flip-flops) of the **synthesis top**, driven by the active simulation set's testbench. So:

- they can only start after synthesis (or implementation) has finished, which failed here;
- the testbench must instantiate the *synthesis top* and use only its pins. `tb_xbar` instantiates `xbar`, which is not the top and does not exist as a module in the netlist of a board design;
- the netlist has no parameters and no internal names. Most testbenches of this project look inside the design (`dut.u_core...`) or shorten time with parameters (`US_PER_MS`). They are made for behavioural simulation only.

The project therefore includes **`tb_zed2_gate`**, a testbench that drives `zed2_top_standalone` through its pins only, at real speed. It is the one to use for post-synthesis and post-implementation simulation.

3. If you want to fix your own project instead

1. **Design Sources:** *Add Sources -> Add or create design sources -> Add Directories*, choose `rtl` and tick *Scan and add RTL include files* and *Add sources from subdirectories*. Also add `boards/zedboard/zed2_top_standalone.v` and `boards/zedboard/zed2_pads.v`. **Untick "Copy sources into project"**.
2. A file that is already in Simulation Sources only: select it, open *Source File Properties*, and tick **Synthesis** and **Implementation** under *Used In* (or remove it and add it again as a design source).
3. **Constraints:** *Add Sources -> Add or create constraints*: `boards/zedboard/zed2_pins.xdc` and `boards/zedboard/zed2_standalone.xdc`.
4. **Top:** in Design Sources right-click `zed2_top_standalone` -> **Set as Top**. `ssc2_axi_top`, `ssc2_axi_bram` and `ssc2_boot` must not be the top.
5. **Simulation Sources:** add `sim/tb_zed2_gate.v` and all of `sim/models/*.v`, then right-click `tb_zed2_gate` -> Set as Top.
6. Settings -> Simulation -> Simulation tab: set *xsim.simulate.runtime* to `all`, so the testbench runs until it prints its result.

4. Running everything

In `build\gui\ssc2_gui.xpr` (or your fixed project):

Flow Navigator	what it does	what success looks like
Run Simulation -> Run Behavioral Simulation	simulates the Verilog of the active simulation set	Tcl Console: `ALL n CHECKS PASSED`
Run Synthesis	Verilog -> netlist for `zed2_top_standalone`	"Synthesis Completed", no errors (warnings are normal)
Run Simulation -> Run Post-Synthesis Functional Simulation	the synthesised netlist with `tb_zed2_gate` (sim_1 active)	`ALL 11 CHECKS PASSED`
Run Simulation -> Run Post-Synthesis Timing Simulation	the same, with estimated gate delays	`ALL 11 CHECKS PASSED`
Run Implementation	place and route (with physical optimisation after placement and after routing, set by the script)	"Implementation Completed"; Open Implemented Design -> Report Timing Summary: WNS >= 0 (if not: TIMING.md)
Run Simulation -> Run Post-Implementation Functional / Timing Simulation	the routed netlist with real delays	`ALL 11 CHECKS PASSED`
Generate Bitstream	`zed2_top_standalone.bit`	then Open Hardware Manager -> Program Device

Notes:

- `tb_zed2_gate` simulates 2 ms of board time: a few seconds as behavioural simulation, minutes on the netlist, and longest (up to tens of minutes) as post-implementation timing simulation, which models every gate and wire delay of about 20 000 LUTs.
- **Other testbenches:** in the Sources window, under Simulation Sources, right-click a simulation set (`sim_xbar`, `sim_system`, `sim_se_programs`, `sim_noc_equiv`, ...) -> **Make Active**, then Run Behavioral Simulation. `sim_noc_equiv` (`tb_noc_router_equiv`) is the check of the timing fix: the new router against the old one, clock by clock. Switch back to `sim_1` before a post-synthesis or post-implementation simulation.
- To look at waveforms: after launching, type `log_wave -r /` and then `restart` and `run all` in the Tcl Console (the default run does not log signals that were not in the wave window).
- Timing is normally proven by **static timing analysis** (Report Timing Summary after implementation, WNS >= 0 at 100 MHz) rather than by timing simulation. Post-implementation timing simulation is a cross-check that the routed design still passes the same test. The first implementation missed 100 MHz by 0.687 ns in the NoC router; the cause and the fix are in TIMING.md.

5. Way 2 (ARM + FPGA)

For Way 2 the top is a block design with the Zynq PS, so it is not made by hand either: run `windows\3_build_way2_arm_fpga.bat` (or `source vivado/build_ps_system.tcl`). It builds the block design, the bitstream and the `.xsa` for Vitis, and leaves the project `build\ps_system\ssc2_ps.xpr` for the GUI. The ARM-side behaviour is verified in simulation by `sim_system` (`tb_ssc2_system`), which drives `ssc2_axi_top` over AXI exactly like the C driver does.